Writing Data from Spark

In this train, we will harness the power of Apache Spark to execute fundamental data extraction and parsing tasks, learning how to manage and transform data using Spark's robust framework.

Learning Objectives

By the end of this train, you should be able to:

- Extract and parse data using Spark.

Extraction and Parsing

Extraction is the process of ingesting the data from a source system (in this case, just a CSV from Kaggle). Parsing, on the other hand, is transforming the data into an appropriate structure to allow compatibility with destination systems. This means changing data into the correct types for your destination and finally writing to a destination system.

```
# Import Spark and some auxiliary functions and set up a SparkSession.


from pyspark import SparkContext
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

sc = SparkContext.getOrCreate()
spark = SparkSession(sc)
```

In this train, we will be using a dataset that can be downloaded from Kaggle. This dataset contains hourly weather data from 25 weather stations across Ireland, obtained from the Irish Meteorological Service, Met Éireann.

```
# Read into Spark using the SparkSession and DataFrameReader only with header
inferred.
# Change the below location to match your current working directory.


raw_df = spark.read.csv('./data/weather_data/hrly_Irish_weather.csv',
header=True)
```

```
python
raw_df.printSchema()
```

```

raw_df.printSchema()
```

The `printSchema()` method gives you a breakdown of the schema of the incoming data, detailing the field name, its type, and whether it is nullable or not.

```
python
raw_df.dropna().show(10)
```

```
+------+---------+--------+---------+-----------------+----+----+----+-----+----
-+----+------+----+-----+---+-----+----+-----+
|county|  station|latitude|longitude|
date|rain|temp|wetb|dewpt|vappr|rhum|   msl|wdsp|wddir|sun|  vis|clht|clamt|
+------+---------+--------+---------+-----------------+----+----+----+-----+----
-+----+------+----+-----+---+-----+----+-----+
|  Mayo|BELMULLET|  54.228|  -10.007|01-jan-1990 00:00| 0.0| 7.6| 7.1|  6.5|
9.7|  93|1003.0|  10|  200|0.0|26000|  16|    8|
|  Mayo|BELMULLET|  54.228|  -10.007|01-jan-1990 01:00| 0.0| 7.6| 7.2|  6.7|
9.8|  94|1003.0|  10|  190|0.0|19000|  18|    8|
|  Mayo|BELMULLET|  54.228|  -10.007|01-jan-1990 02:00| 0.0| 7.8| 7.2|  6.5|
9.7|  91|1003.1|   8|  200|0.0|22000|  18|    7|
|  Mayo|BELMULLET|  54.228|  -10.007|01-jan-1990 03:00| 0.0| 7.7| 6.8|  5.7|
9.2|  87|1003.4|  12|  220|0.0|26000|  18|    6|
|  Mayo|BELMULLET|  54.228|  -10.007|01-jan-1990 04:00| 0.5| 7.5| 6.5|  5.3|
8.9|  86|1003.2|  13|  210|0.0|30000| 999|    3|
|  Mayo|BELMULLET|  54.228|  -10.007|01-jan-1990 05:00| 0.0| 6.8| 5.9|  4.7|
8.6|  87|1003.9|   9|  220|0.0|30000| 999|    3|
|  Mayo|BELMULLET|  54.228|  -10.007|01-jan-1990 06:00| 0.0| 5.8| 4.9|  3.7|
7.9|  86|1004.1|   7|  220|0.0|30000| 999|    3|
|  Mayo|BELMULLET|  54.228|  -10.007|01-jan-1990 07:00| 0.0| 5.3| 4.6|  3.6|
7.9|  89|1004.7|   8|  220|0.0|30000| 999|    4|
|  Mayo|BELMULLET|  54.228|  -10.007|01-jan-1990 08:00| 0.0| 6.0| 5.4|  4.6|
8.5|  91|1004.9|  10|  190|0.0|30000| 999|    2|
|  Mayo|BELMULLET|  54.228|  -10.007|01-jan-1990 09:00| 0.0| 6.1| 5.4|  4.5|
8.4|  89|1005.4|  12|  200|0.1|20000| 999|    3|
+------+---------+--------+---------+-----------------+----+----+----+-----+----
-+----+------+----+-----+---+-----+----+-----+
only showing top 10 rows
```

This does give us a good idea, but it's not good enough, yet. Let's use the `describe()` method to get summary statistics of the DataFrame.

▼Did you know? ✖

If the DataFrame has too many columns to display using the built-in Spark viewer, use the `toPandas()` method to view the DataFrame using the Pandas interface. It is important, however, to note that you should limit the size of the DataFrame in such a case as to not pull a massive DataFrame into memory, causing your computer to crash.

From that, let's define the schema. Many of the fields are numerical values. Spark has **Integer** types, **Decimal** types, and **Float** types, along with various others, which we have dealt with elsewhere. Parsing the DataFrame, we are not going to parse the date as a

timestamp or **DateTime** just yet. As of the writing of this train, Spark does not have a very accurate **DateTime** or **timestamp** parser when reading DataFrames. As such, it is better to parse the **DateTime** where we can ensure that the coercion was done correctly.

```
from pyspark.sql.types import StructField, StructType, StringType, IntegerType,
FloatType, TimestampType

weather_schema = StructType([
    StructField('county', StringType()),
    StructField('station', StringType()),
    StructField('latitude', FloatType()),
    StructField('longitude', FloatType()),
    StructField('date', StringType()), # While this is a date, it will likely
not coerce correctly. Let's parse manually.
    StructField('rain', FloatType()),
    StructField('temp', FloatType()),
    StructField('wetb', FloatType()),
    StructField('dewpt', FloatType()),
    StructField('vappr', FloatType()),
    StructField('rhum', IntegerType()),
    StructField('msl', FloatType()),
    StructField('wdsp', IntegerType()),
    StructField('wddir', IntegerType()),
    StructField('sun', FloatType()),
    StructField('vis', IntegerType()),
    StructField('clht', IntegerType()),
    StructField('clamt', IntegerType())
])
```

```
# Ensure that the below path matches your current working directory.


typed_df = spark.read.csv('./data/weather_data/hrly_Irish_weather.csv',
header=True, schema=weather_schema)
```

Let's parse the date column using the `to_timestamp()` function, which takes the field to convert as the first argument and the format as the second (refer to [ISO 8601] for date conventions). This will allow us to use all kinds of fancy methods on the `date` column.

```
typed_df = typed_df.withColumn('date', F.to_timestamp(F.col('date'), format="dd-
MMM-yyy HH:mm"))
```

Running `printSchema()` will print the schema as output.

```
typed_df.printSchema()
```

Running this will show the first 10 rows of the dataframe.

```
typed_df.show(10)
```

```
+------+-------+--------+---------+-------------------+----+----+----+-----+----
-+----+------+----+-----+----+----+----+-----+
|county|station|latitude|longitude|
date|rain|temp|wetb|dewpt|vappr|rhum|   msl|wdsp|wddir| sun| vis|clht|clamt|
+------+-------+--------+---------+-------------------+----+----+----+-----+----
-+----+------+----+-----+----+----+----+-----+
|Galway|ATHENRY|  53.289|   -8.786|2011-06-26 01:00:00| 0.0|15.3|14.5| 13.9|
15.8|  90|1016.0|   8|  190|null|null|null| null|
|Galway|ATHENRY|  53.289|   -8.786|2011-06-26 02:00:00| 0.0|14.7|13.7| 12.9|
14.9|  89|1015.8|   7|  190|null|null|null| null|
|Galway|ATHENRY|  53.289|   -8.786|2011-06-26 03:00:00| 0.0|14.3|13.4| 12.6|
14.6|  89|1015.5|   6|  190|null|null|null| null|
|Galway|ATHENRY|  53.289|   -8.786|2011-06-26 04:00:00| 0.0|14.4|13.6| 12.8|
14.8|  90|1015.3|   7|  180|null|null|null| null|
|Galway|ATHENRY|  53.289|   -8.786|2011-06-26 05:00:00| 0.0|14.4|13.5| 12.7|
14.7|  89|1015.1|   6|  190|null|null|null| null|
|Galway|ATHENRY|  53.289|   -8.786|2011-06-26 06:00:00| 0.0|14.9|13.8| 13.0|
15.0|  88|1015.0|   8|  180|null|null|null| null|
|Galway|ATHENRY|  53.289|   -8.786|2011-06-26 07:00:00| 0.0|15.1|13.5| 12.2|
14.2|  83|1014.9|   8|  180|null|null|null| null|
|Galway|ATHENRY|  53.289|   -8.786|2011-06-26 08:00:00| 0.0|15.2|13.0| 11.1|
13.2|  76|1014.7|   7|  170|null|null|null| null|
|Galway|ATHENRY|  53.289|   -8.786|2011-06-26 09:00:00| 0.0|16.1|13.2| 10.8|
12.9|  70|1014.4|   7|  160|null|null|null| null|
|Galway|ATHENRY|  53.289|   -8.786|2011-06-26 10:00:00| 0.0|16.8|13.3| 10.3|
12.5|  65|1014.1|  10|  180|null|null|null| null|
+------+-------+--------+---------+-------------------+----+----+----+-----+----
-+----+------+----+-----+----+----+----+-----+
only showing top 10 rows
```

If we needed this data in a pandas dataframe we can run:

```
typed_df.describe().toPandas()
```

Finally, let's output this to a destination. In this notebook, we will use `parquet` in the same directory. This has various advantages, the first of which is that it allows us to read it in again while maintaining the fields, field names, and other metadata.

```
# Ensure that the below path matches your current working directory.


typed_df.write.parquet('./data/weather_data/hrly_Irish_weather')
```

Conclusion

Spark facilitates efficient data extraction, parsing, and writing, enabling the transformation of raw data into structured formats suitable for various destination systems.

▼Key Takeaways ✖

- **Takeaway 1:** Extraction involves reading data from a source system, as demonstrated by loading a CSV file into a Spark DataFrame.
- **Takeaway 2:** Parsing transforms the data into an appropriate structure, including data type conversions, exemplified by parsing the 'date' column into a timestamp format using `to_timestamp()`.
- **Takeaway 3:** Spark supports writing data to various formats, with Parquet being advantageous for preserving schema and metadata, as shown by the `typed_df.write.parquet()` method.